MZ2POL-00: SDK Management Zone Analysis Tool

> **Series:** MZ2POL — Management Zone Migration: Segments, Policies, Workflows | **Notebook:** 1 of 15 | **Created:** December 2025 | **Last Updated:** 10/13/2026
>
> **Purpose:** Export every management-zone rule, measure how each zone is used, and check whether the **data** — not just the entities — carries the context the replacements need. The output feeds MZ2POL-03 (classification), MZ2POL-11 (rule translation) and the MZ2POL-14 workshop.

What This Notebook Does

1. **Export all zone rules** — every `ME`, `DIMENSION` and `SELECTOR` rule, one row per condition, with all eight propagation flags
2. **Analyze rule patterns** — rule types, entity types, condition keys, and which conditions need special translation
3. **Measure zone usage** — query activity per zone over 30 days, the evidence for deleting unused zones
4. **Count entities per zone** — how much each zone covers
5. **Check security context on the data** — per table, type (string or array), and whether entity contexts are only copied from zone names
6. **Find tag patterns** — the classic tag vocabulary the zones were built from
7. **List Kubernetes namespaces and host groups** — the built-in fields many zones translate to
8. **Summarize readiness** — one table combining the above

Table of Contents

1. Export All Management Zone Rules

2. MZ Rule Pattern Analysis
3. Zone Usage
4. Entity Counts by Management Zone
5. Security Context on the Data
6. Tag Pattern Analysis
7. Kubernetes Namespace Analysis
8. Host Group Analysis
9. Migration Readiness Summary
10. Related Notebooks
11. Documentation

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with management zones still configured
Permissions	`settings:objects:read` to read `builtin:management-zones` (TypeScript cells); `storage:entities:read`, `storage:logs:read`, `storage:spans:read`, `storage:events:read`, `storage:metrics:read` for the DQL cells
Runtime	A Dynatrace notebook — the TypeScript cells use `@dynatrace-sdk/client-classic-environment-v2` and `@dynatrace-sdk/client-query`
Next	Classify the output with MZ2POL-03, then translate each rule with MZ2POL-11

1. Export All Management Zone Rules

This function reads `builtin:management-zones` through the Settings API and returns one row per rule condition. The schema defines three rule types — `ME` (monitored entity),

`DIMENSION` (dimensional data: logs and metrics) and `SELECTOR` (entity selector) — and each stores its logic in a different property, so each is flattened separately:

Rule type	Where the logic lives	Row per
`ME`	`attributeRule`: entity type, conditions, eight propagation flags	Condition
`DIMENSION`	`dimensionRule`: `appliesTo` (`ANY`, `LOG`, `METRIC`) and conditions of type `DIMENSION`, `LOG_FILE_NAME` or `METRIC_KEY`	Condition
`SELECTOR`	`entitySelector` string	Rule

Condition values can be text, an enum, an integer, an entity ID or a tag; the export takes whichever is set. The `propagation` column lists every propagation flag that is switched on — MZ2POL-10 §7 explains what replaces each one.

> **Sources:** Management zones settings schema (DT docs) — rule `type` enum `ME`, `DIMENSION`, `SELECTOR`; the `ManagementZoneAttributeRule`, `DimensionRule`, `AttributeCondition` and `DimensionCondition` objects.

```
import { settingsObjectsClient } from "@dynatrace-sdk/client-classic-
environment-v2";

// Shape of builtin:management-zones objects, per the Settings schema table.
interface AttributeCondition {
  key: string;
  operator: string;
  dynamicKey?: string;
  stringValue?: string;
  enumValue?: string;
  integerValue?: number;
  entityId?: string;
  tag?: string;
  caseSensitive?: boolean;
}

interface AttributeRule {
  entityType: string;
  conditions?: AttributeCondition[];
  serviceToHostPropagation?: boolean;
  serviceToPGPropagation?: boolean;
  pgToHostPropagation?: boolean;
  pgToServicePropagation?: boolean;
  hostToPGPropagation?: boolean;
  customDeviceGroupToCustomDevicePropagation?: boolean;
  azureToServicePropagation?: boolean;
  azureToPGPropagation?: boolean;
}

interface DimensionCondition {
  conditionType: string; // DIMENSION | LOG_FILE_NAME | METRIC_KEY
  key?: string;
  ruleMatcher: string;   // BEGINS_WITH | EQUALS
  value: string;
}

interface MzRule {
  enabled: boolean;
  type: "ME" | "DIMENSION" | "SELECTOR";
  attributeRule?: AttributeRule;
  dimensionRule?: { appliesTo: string; conditions?: DimensionCondition[] };
  entitySelector?: string;
}

interface MzDefinition {
  name: string;
  description?: string;
```

```
  rules?: MzRule[];
}

const PROPAGATION_FLAGS = [
  "serviceToHostPropagation", "serviceToPGPropagation", "pgToHostPropagation",
"pgToServicePropagation",
  "hostToPGPropagation", "customDeviceGroupToCustomDevicePropagation",
"azureToServicePropagation", "azureToPGPropagation",
] as const;

// Fetch every zone, following nextPageKey (other parameters must be omitted
on later pages).
async function getAllZones(): Promise&lt;MzDefinition[]&gt; {
  const zones: MzDefinition[] = [];
  let resp = await settingsObjectsClient.getSettingsObjects({
    schemaIds: "builtin:management-zones", pageSize: 100, fields:
"objectId,value",
  });
  zones.push(...resp.items.map((o) =&gt; o.value as MzDefinition));
  while (resp.nextPageKey) {
    resp = await settingsObjectsClient.getSettingsObjects({ nextPageKey:
resp.nextPageKey });
    zones.push(...resp.items.map((o) =&gt; o.value as MzDefinition));
  }
  return zones;
}

function conditionValue(c: AttributeCondition): string {
  if (c.tag !== undefined) return c.tag;
  if (c.stringValue !== undefined) return c.stringValue;
  if (c.enumValue !== undefined) return c.enumValue;
  if (c.integerValue !== undefined) return String(c.integerValue);
  if (c.entityId !== undefined) return c.entityId;
  return "";
}

// One record per condition (ME and DIMENSION) or per selector (SELECTOR).
function flatten(zones: MzDefinition[]) {
  const records: Record&lt;string, string | number | boolean&gt;[] = [];
  zones.forEach((zone) =&gt; {
    (zone.rules ?? []).forEach((rule, ruleIndex) =&gt; {
      const base = { zone: zone.name, rule: ruleIndex + 1, ruleType:
rule.type, enabled: rule.enabled };
      if (rule.type === "ME" &amp;&amp; rule.attributeRule) {
        const attr = rule.attributeRule;
        const propagation = PROPAGATION_FLAGS.filter((f) =&gt; attr[f] ===
true).join(",");
        const conditions = attr.conditions ?? [];
```

```
        if (conditions.length === 0) {
          records.push({ ...base, appliesTo: attr.entityType, key: "",
operator: "", value: "", caseSensitive: false, propagation });
        }
        conditions.forEach((c) =&gt; records.push({
          ...base, appliesTo: attr.entityType,
          key: c.dynamicKey ? `${c.key}:${c.dynamicKey}` : c.key,
          operator: c.operator, value: conditionValue(c),
          caseSensitive: c.caseSensitive ?? false, propagation,
        }));
      } else if (rule.type === "DIMENSION" &amp;&amp; rule.dimensionRule) {
        const dim = rule.dimensionRule;
        (dim.conditions ?? []).forEach((c) =&gt; records.push({
          ...base, appliesTo: dim.appliesTo,
          key: c.key ? `${c.conditionType}:${c.key}` : c.conditionType,
          operator: c.ruleMatcher, value: c.value, caseSensitive: false,
propagation: "",
        }));
      } else if (rule.type === "SELECTOR") {
        records.push({
          ...base, appliesTo: "", key: "entitySelector", operator: "SELECTOR",
          value: rule.entitySelector ?? "", caseSensitive: false, propagation:
"",
        });
      }
    });
  });
  return records;
}

export default async function () {
  return flatten(await getAllZones());
}
```

---

2. MZ Rule Pattern Analysis

The same export, counted. The `Translation` rows flag the conditions that need more than a field lookup — read them with MZ2POL-11:

Row	What it means	MZ2POL-11
`exclusion (NOT_*)`	Segments and record-level IAM conditions cannot exclude; push the `NOT` into an enrichment rule	§5
`regex / numeric / IP range`	No Grail surface evaluates these; compute the value at the source	§4
`tag-based condition`	Promote the tag to `primary_tags.*` or `dt.security_context`	§3
`host group → built-in field`	`dt.host_group.id` already holds the value	§2, step 1
`Propagation flag` rows	Decide where the value is really true instead of copying the flag	MZ2POL-10 §7
`Dimension condition` rows	Dimensional rules are nearly 1:1 — the field is on the record	§6

```
import { settingsObjectsClient } from "@dynatrace-sdk/client-classic-
environment-v2";

interface MzRule {
  enabled: boolean;
  type: "ME" | "DIMENSION" | "SELECTOR";
  attributeRule?: { entityType: string; conditions?: { key: string; operator:
string }[] } &amp; Record&lt;string, unknown&gt;;
  dimensionRule?: { appliesTo: string; conditions?: { conditionType: string }
[] };
  entitySelector?: string;
}
interface MzDefinition { name: string; rules?: MzRule[] }

const PROPAGATION_FLAGS = [
  "serviceToHostPropagation", "serviceToPGPropagation", "pgToHostPropagation",
"pgToServicePropagation",
  "hostToPGPropagation", "customDeviceGroupToCustomDevicePropagation",
"azureToServicePropagation", "azureToPGPropagation",
];
// Operators no Grail surface evaluates directly (MZ2POL-11 §4): compute these
at the source.
const HARD_OPERATORS = ["REGEX_MATCHES", "NOT_REGEX_MATCHES",
"IS_IP_IN_RANGE", "NOT_IS_IP_IN_RANGE",
  "GREATER_THAN", "NOT_GREATER_THAN", "GREATER_THAN_OR_EQUAL",
"NOT_GREATER_THAN_OR_EQUAL",
  "LOWER_THAN", "NOT_LOWER_THAN", "LOWER_THAN_OR_EQUAL",
"NOT_LOWER_THAN_OR_EQUAL"];

async function getAllZones(): Promise&lt;MzDefinition[]&gt; {
  const zones: MzDefinition[] = [];
  let resp = await settingsObjectsClient.getSettingsObjects({
    schemaIds: "builtin:management-zones", pageSize: 100, fields:
"objectId,value",
  });
  zones.push(...resp.items.map((o) =&gt; o.value as MzDefinition));
  while (resp.nextPageKey) {
    resp = await settingsObjectsClient.getSettingsObjects({ nextPageKey:
resp.nextPageKey });
    zones.push(...resp.items.map((o) =&gt; o.value as MzDefinition));
  }
  return zones;
}

export default async function () {
  const zones = await getAllZones();
  const count: Record&lt;string, number&gt; = {};
```

```
  const add = (k: string, n = 1) =&gt; { count[k] = (count[k] ?? 0) + n; };

  zones.forEach((zone) =&gt; {
    (zone.rules ?? []).forEach((rule) =&gt; {
      add(`Rule type · ${rule.type}`);
      if (rule.type === "ME" &amp;&amp; rule.attributeRule) {
        const attr = rule.attributeRule;
        add(`ME entity type · ${attr.entityType}`);
        PROPAGATION_FLAGS.forEach((f) =&gt; { if (attr[f] === true)
add(`Propagation flag · ${f}`); });
        (attr.conditions ?? []).forEach((c) =&gt; {
          add(`Condition key · ${c.key}`);
          if (c.operator.startsWith("NOT_")) add("Translation · exclusion
(NOT_*) → push into enrichment");
          if (HARD_OPERATORS.includes(c.operator)) add("Translation · regex /
numeric / IP range → compute at source");
          if (c.key.endsWith("_TAGS")) add("Translation · tag-based
condition");
          if (c.key === "HOST_GROUP_NAME" || c.key === "HOST_GROUP_ID")
add("Translation · host group → built-in field");
        });
      } else if (rule.type === "DIMENSION" &amp;&amp; rule.dimensionRule) {
        (rule.dimensionRule.conditions ?? []).forEach((c) =&gt; add(`Dimension
condition · ${c.conditionType}`));
      }
    });
  });

  const rows = [
    { category: "Summary", metric: "Management zones", count: zones.length },
    { category: "Summary", metric: "Rules", count: zones.reduce((n, z) =&gt; n
+ (z.rules?.length ?? 0), 0) },
    ...Object.entries(count).map(([k, n]) =&gt; {
      const [category, metric] = k.split(" · ");
      return { category, metric, count: n };
    }),
  ];
  return rows.sort((a, b) =&gt; a.category.localeCompare(b.category) ||
b.count - a.count);
}
```

3. Zone Usage

Dynatrace self-monitoring records how often each zone is used in queries. A zone with no activity over 30 days is a delete candidate, not a translation job (MZ2POL-14, Session 1). Combine this with role assignments, alerting profiles and dashboards before deciding — a zone can be unqueried and still scope a role or a profile.

```
// Query activity per management zone over 30 days. Zones absent from the
result had none.
timeseries q = sum(dt.sfm.server.management_zones.queries_counter), from:-30d,
by:{dt.management_zone.name}
| fieldsAdd queries_30d = arraySum(q)
| fieldsRemove q, timeframe, interval
| sort queries_30d desc
| limit 200
```

On the validation tenant (10/13/2026) this returned per-zone counts ranging from about 128,000 queries down to a single query in 30 days. The metric is a self-monitoring metric observed on that tenant rather than a documented API; if it returns nothing on yours, check it exists with `metrics | filter metric.key == "dt.sfm.server.management_zones.queries_counter"` before treating the result as "no usage".

---

4. Entity Counts by Management Zone

How much each zone covers — useful for sizing, not for prioritizing. Priority follows risk (MZ2POL-03): a small zone guarding regulated data comes before a large filtering zone.

Services per Management Zone

```
// Services per management zone — sizing, not priority.
fetch dt.entity.service
| expand mz = managementZones
| filter isNotNull(mz)
| summarize entityCount = count(), by: {managementZone = mz}
| sort entityCount desc
| limit 30

// Note: dt.entity.* is deprecated, but this is a migration-ASSESSMENT query — it inspects
// the classic constructs this migration replaces, so keep the classic query above:
//   - management zones have NO Smartscape node equivalent; they are what this migration
//     replaces with segments and policies.
```

---

5. Security Context on the Data

Earlier versions of this notebook measured `dt.security_context` on service and host **entities**. That number is misleading for a migration: the classic *Grail Security Context* mapping can copy zone names onto entities, and Dynatrace states that data records *"do not inherit the management zones of that entity."* On the validation tenant, entity coverage read 100% (27 of 27 services, 23 of 23 hosts) while about 44% of logs and 67% of events carried no security context. IAM record conditions test the **record**, so measure the records.

Check A — which tables hold the field as an array? An `array` row means `=`, `IN` and `startsWith` policy conditions never match those records; use `MATCH` (MZ2POL-12 §6).

```
// Type of dt.security_context per table.
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| fieldsAdd sc_type = type(dt.security_context)
| summarize {records = count()}, by:{sc_type}
| fieldsAdd table = "logs"
| append [fetch spans, from:-1h | filter isNotNull(dt.security_context) |
fieldsAdd sc_type = type(dt.security_context) | summarize {records = count()},
by:{sc_type} | fieldsAdd table = "spans"]
| append [fetch events, from:-24h | filter isNotNull(dt.security_context) |
fieldsAdd sc_type = type(dt.security_context) | summarize {records = count()},
by:{sc_type} | fieldsAdd table = "events"]
| fields table, sc_type, records
```

Check B — what share of each table has no security context? These records are invisible to any user scoped only by `dt.security_context`.

```
// Records without a security context, per table.
fetch logs, from:-1h
| summarize {records = count(), without_sc =
countIf(isNull(dt.security_context))}
| fieldsAdd table = "logs"
| append [fetch spans, from:-1h | summarize {records = count(), without_sc =
countIf(isNull(dt.security_context))} | fieldsAdd table = "spans"]
| append [fetch events, from:-1h | summarize {records = count(), without_sc =
countIf(isNull(dt.security_context) or arraySize(dt.security_context) == 0)} |
fieldsAdd table = "events"]
| fieldsAdd without_pct = round(100.0 * without_sc / records, decimals: 1)
| fields table, records, without_sc, without_pct
```

Check C — are entity security contexts just copies of zone names? If yes, they scope classic entity queries today but stop matching scoped settings as the Smartscape transition arrives (MZ2POL-12 §5), and they say nothing about the data.

```
// Classic host entities whose security context equals one of their zone
names. Zero bytes scanned.
fetch dt.entity.host
| fields entity.name, managementZones, dt.security_context
| fieldsAdd sc_from_mz =
arraySize(arrayRemoveNulls(iCollectArray(if(in(dt.security_context[],
managementZones), dt.security_context[])))) &gt; 0
| summarize {hosts = count(),
    with_sc = countIf(arraySize(dt.security_context) &gt; 0),
    sc_matches_an_mz = countIf(sc_from_mz)}
```

On the validation tenant (10/13/2026): logs and spans held a string and events an array (Check A); about 44% of logs and 67% of events had no security context (Check B); all 23 hosts had an entity security context matching a zone name (Check C). The fix for Checks A–C is tagging at source — MZ2POL-10.

> **Sources:** Grant access to entities with security context (DT docs) — *"Logs, spans, metrics, and events powered by Grail that are sent from an entity do not inherit the management zones of that entity."*, Permissions in Grail (DT docs) — *"Using = , STARTSWITH or IN when the field holds an array will always return false ."*

---

6. Tag Pattern Analysis

The classic tag vocabulary the zones were built from. Each key is a candidate for a built-in field, a primary tag, a special field or retirement (MZ2POL-10 §2, MZ2POL-11 §3). Classic auto-tags appear only here — they never reach Grail data.

Most Common Host Tags

```
// Most common classic host tags (key:value).
fetch dt.entity.host
| expand tag = tags
| filter isNotNull(tag)
| summarize count = count(), by: {tag}
| sort count desc
| limit 50

// Note: dt.entity.* is deprecated, but this is a migration-ASSESSMENT query — it inspects
// the classic constructs this migration replaces, so keep the classic query above:
//   - entity tags are not a flat "tags" field on Smartscape (resolve via getNodeField).
```

Tag Key Frequency

Extract the key from the `key:value` format to see which tag keys are most common.

```
// Classic tag keys and how many distinct values each has.
fetch dt.entity.host
| expand tag = tags
| filter isNotNull(tag)
| filter contains(tag, ":")
| parse tag, "LD:tagKey ':' LD:tagValue"
| summarize {count = count(), uniqueValues = countDistinct(tagValue)}, by:
{tagKey}
| sort count desc
| limit 30

// Note: dt.entity.* is deprecated, but this is a migration-ASSESSMENT query —
it inspects
// the classic constructs this migration replaces, so keep the classic query
above:
//   - entity tags are not a flat "tags" field on Smartscape (resolve via
getNodeField).
```

---

7. Kubernetes Namespace Analysis

Namespace zones translate to the built-in `k8s.namespace.name` field — no tagging work (MZ2POL-11 §3).

```
// Kubernetes namespaces — candidates for the built-in k8s.namespace.name
field.
fetch dt.entity.cloud_application_namespace
| fields namespace = entity.name, id
| sort namespace asc
| limit 30

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_NAMESPACE"
//   | fields namespace = name, id
//   | sort namespace asc
//   | limit 30
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
```

---

8. Host Group Analysis

Host-group zones translate to the built-in `dt.host_group.id` field. To see which zones are really host-group zones — and which contain hosts with no host group at all — run MZ2POL-11 §9, Check 1.

```
// Host groups — candidates for the built-in dt.host_group.id field.
fetch dt.entity.host_group
| fields hostGroup = entity.name, id

// Note: dt.entity.* is deprecated. There is no Smartscape "HOST_GROUP" node
type
// (smartscapeNodes "HOST_GROUP" returns nothing); host-group membership is
exposed as the
// dt.host_group.id field on HOST nodes (e.g. smartscapeNodes "HOST" |
summarize by:{dt.host_group.id}).
// Keep the classic query above to list host-group entities.
```

---

9. Migration Readiness Summary

One table: how many zones exist, how many were queried in the last 30 days, and how much of each table lacks a security context. It reads the zone count from the response's `totalCount` — the `fields` parameter selects object fields (`objectId`, `value`, …) and does not accept `totalCount`.

```
import { settingsObjectsClient } from "@dynatrace-sdk/client-classic-
environment-v2";
import { queryExecutionClient, type ResultRecord } from "@dynatrace-
sdk/client-query";

async function zoneCount(): Promise&lt;number&gt; {
  // totalCount is part of every response; "fields" only selects object
fields.
  const resp = await settingsObjectsClient.getSettingsObjects({
    schemaIds: "builtin:management-zones", pageSize: 1, fields: "objectId",
  });
  return resp.totalCount;
}

async function runDQL(query: string): Promise&lt;ResultRecord[]&gt; {
  // queryExecute returns a requestToken instead of the result when the query
outlasts
  // requestTimeoutMilliseconds. Poll to a final state, and fail loudly rather
than
  // returning [] — an empty result here would read as "nothing to fix".
  const started = await queryExecutionClient.queryExecute({
    body: { query, requestTimeoutMilliseconds: 60000, maxResultRecords: 50 },
  });
  let resp = started;
  while (resp.state === "NOT_STARTED" || resp.state === "RUNNING") {
    resp = await queryExecutionClient.queryPoll({ requestToken:
started.requestToken!, requestTimeoutMilliseconds: 30000 });
  }
  if (resp.state !== "SUCCEEDED" || !resp.result) {
    throw new Error(`DQL query did not succeed: ${resp.state}`);
  }
  return resp.result.records.filter((r): r is ResultRecord =&gt; r !== null);
}

export default async function () {
  const zones = await zoneCount();

  // Security context on the DATA, per table (MZ2POL-12 §10, Check 2).
  const coverage = await runDQL(`
    fetch logs, from:-1h
    | summarize {records = count(), without_sc =
countIf(isNull(dt.security_context))}
    | fieldsAdd table = "logs"
    | append [fetch spans, from:-1h | summarize {records = count(), without_sc
= countIf(isNull(dt.security_context))} | fieldsAdd table = "spans"]
    | append [fetch events, from:-1h | summarize {records = count(),
without_sc = countIf(isNull(dt.security_context) or
```

```
arraySize(dt.security_context) == 0)} | fieldsAdd table = "events"]
  `);

  // Zones with no query activity in 30 days — delete candidates (MZ2POL-14,
Session 1).
  const usage = await runDQL(`
    timeseries q = sum(dt.sfm.server.management_zones.queries_counter),
from:-30d, by:{dt.management_zone.name}
    | fieldsAdd queries_30d = arraySum(q)
    | summarize {zones_with_queries = countIf(queries_30d &gt; 0)}
  `);

  const rows: { category: string; metric: string; value: number | string;
status: string }[] = [
    { category: "Zones", metric: "Management zones configured", value: zones,
status: "INFO" },
    {
      category: "Zones", metric: "Zones queried in the last 30 days",
      value: Number(usage[0]?.zones_with_queries ?? 0),
      status: Number(usage[0]?.zones_with_queries ?? 0) &lt; zones ? "DELETE
CANDIDATES EXIST" : "OK",
    },
  ];
  coverage.forEach((r) =&gt; {
    const records = Number(r.records ?? 0);
    const without = Number(r.without_sc ?? 0);
    const pct = records &gt; 0 ? Math.round((1000 * without) / records) / 10 :
0;
    rows.push({
      category: `Data · ${String(r.table)}`, metric: "Records without
dt.security_context (1 h)",
      value: `${without} of ${records} (${pct}%)`, status: records === 0 ? "NO
DATA" : pct &gt; 0 ? "NEEDS_WORK" : "GOOD",
    });
  });
  return rows;
}
```

Next Steps

> **This tool is an inventory aid, not a converter.** There is **no automatic Management Zone to Segment conversion** in Dynatrace — no button, no API, no

> wizard. Unlike Metric Events, which have a supported automatic upgrade path to Anomaly Detectors, every Management Zone must be re-expressed by hand as a segment or an IAM policy. The analysis tells you *how much work each zone is*; it does not perform the migration.

Based on the analysis above

1. **Zones with no usage (§3)?** → Delete candidates, after checking role assignments and alerting profiles (MZ2POL-14, Session 1).
2. **Records without a security context (§5, Check B)?** → Tagging at source before anything else (MZ2POL-10); the readiness gate in MZ2POL-10 §10.
3. **Entity contexts copied from zone names (§5, Check C)?** → Do not count them as coverage; plan edge-set contexts (MZ2POL-12 §5).
4. **Array-typed security context (§5, Check A)?** → `MATCH` in every policy and boundary (MZ2POL-12 §6).
5. **`Translation` rows in §2?** → Work them through the MZ2POL-11 decision path; the export in §1 is the input to the MZ2POL-11 worksheet (MZ2POL-11 §10).
6. **Host-group or namespace zones (§7, §8)?** → Built-in fields — the cheapest translations (MZ2POL-11 §2, step 1).

Prioritization

Priority follows risk, not size — use the P1–P4 scheme in **MZ2POL-03 §4**: security and compliance access first, then team access and alerting scope, then filtering-only zones; unused zones are deleted.

Related Notebooks

- **MZ2POL-01**: Introduction — Why Migrate (with reading paths)
- **MZ2POL-03**: Assessment and Planning
- **MZ2POL-10**: Tagging at Source
- **MZ2POL-11**: Translating MZ Rules into Source Attributes
- **MZ2POL-12**: IAM Reference Architecture
- **MZ2POL-14**: Zone Disposition Workshop

- **MZ2POL-95**: End-to-End Migration LAB

Documentation

- Management zones settings schema (DT docs)
- Management-zone rules (DT docs)
- Upgrade role-based permissions to Dynatrace IAM policies (DT docs)
- Grant access to entities with security context (DT docs)
- Permissions in Grail (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.